2_Dfs

```
#include <iostream> // For input and output
#include <vector> // For adjacency list representation
#include <stack> // For stack used in DFS
#include <omp.h> // For OpenMP parallel programming
using namespace std;
const int MAX = 100000; // Maximum number of nodes

vector<int> graph[MAX]; // Adjacency list for graph
bool visited[MAX]; // Visited array to track visited nodes

// Function to perform DFS traversal
void dfs(int node)
{
stack<int> s; // Stack used for DFS
s.push(node); // Push starting node into stack
// Continue until stack becomes empty
while (!s.empty())
{
int curr_node = s.top(); // Get top node from stack
s.pop(); // Remove it from stack
// If current node is not visited
if (!visited[curr_node])
{
visited[curr_node] = true; // Mark current node as visited

cout << curr_node << " "; // Print current node

// Check all adjacent nodes in parallel
#pragma omp parallel for
for (int i = 0; i < graph[curr_node].size(); i++)
{
int adj_node = graph[curr_node][i]; // Get adjacent node

// If adjacent node is not visited
if (!visited[adj_node])
{
// Critical section because stack is shared
#pragma omp critical
{
s.push(adj_node); // Push adjacent node into stack
}
}
}
}
}
```

```
}
// Main function
int main()
{
int n, m, start_node; // n = nodes, m = edges,start_node = starting point
cout << "Enter number of nodes, number of edges and starting node:\n";
cin >> n >> m >> start_node;

cout << "Enter pairs of connected nodes:\n";
// Input graph edges
for (int i = 0; i < m; i++)
{
int u, v; // Edge between u and v
cin >> u >> v;

graph[u].push_back(v); // Add v to u's adjacency list
graph[v].push_back(u); // Add u to v's adjacency list because graph is undirected
}
// Initialize all nodes as not visited
#pragma omp parallel for
for (int i = 0; i < n; i++)
{
visited[i] = false;
}
cout << "\nDFS Traversal:\n";

dfs(start_node); // Start DFS from given node
return 0; // End program
}
```